

Taller Microservicios

1. Objetivo general

- Desplegar una arquitectura de software que ilustre algunos patrones de microservicios utilizando la infraestructura de GCP y la tecnología Kong.

2. Reto

- Configurar Kong como API Gateway: registrar los microservicios y crear sus rutas o paths.

3. Marco Conceptual

2.1 Microservicios

Es un estilo de arquitectura que consiste en construir una aplicación en la cual cada servicio es autónomo, desarrolla una capacidad de negocio y se ejecuta en su propio proceso. Estos servicios se comunican entre sí mediante mecanismos livianos y con un mínimo de administración centralizada.

Este estilo de arquitectura permite simplificar los ambientes complejos por medio de la distribución de servicios, una mayor cohesión y menor acoplamiento, a la vez que disminuye la complejidad del despliegue; pero dificulta el mantenimiento de una visión general del negocio y su coordinación.

2.2 Docker

Docker es una tecnología open-source de virtualización diseñada para facilitar la creación, implementación y ejecución de aplicaciones en entornos livianos y portátiles llamados contenedores. Estos contenedores encapsulan de manera eficiente y consistente tanto el código como las dependencias de una aplicación, lo que permite su ejecución sin problemas en cualquier entorno que admita Docker. Al proporcionar un entorno aislado y reproducible para las aplicaciones, Docker simplifica el desarrollo y la administración de software, facilitando la creación de soluciones escalables y distribuibles. Para más información puede consultar el marco conceptual del [taller de Base de datos documental](#) del curso o ingresar a la página oficial de [Docker](#).

2.3 Kong (API Gateway)

Primero que todo, entiéndase que Lua es un lenguaje de programación multiparadigma y multiplataforma diseñado para uso incrustado en las aplicaciones. Kong es una aplicación Lua (habilita scripts Lua o complementos) que se ejecuta en Nginx (servidor Web). Sus desarrolladores consideran a Kong como un “parangón de la arquitectura de microservicio: en su núcleo, implementa la abstracción de la base de datos, el enrutamiento y la

administración de complementos. Los complementos pueden vivir en bases de código separadas e inyectarse en cualquier parte del ciclo de vida de la solicitud, todo en unas pocas líneas de código”.

Si desea obtener información adicional de Kong, puede visitar la documentación en su sitio web:

<https://docs.konghq.com>

4. Contexto

En este taller, va a desplegar dos microservicios que componen la aplicación web de monitoreo que ha venido desarrollando a lo largo del semestre en el laboratorio. El primer microservicio gestiona la creación y consulta de las medidas de monitoreo “measurements”, y el segundo microservicio gestiona la creación y consulta de las variables del sistema de monitoreo. En la figura 1, puede observar el modelo de datos, donde una variable se compone de cero o muchas medidas “measurement”. En este taller, vamos a tomar estos dos como subdominios del sistema, y serán desplegados en dos microservicios.

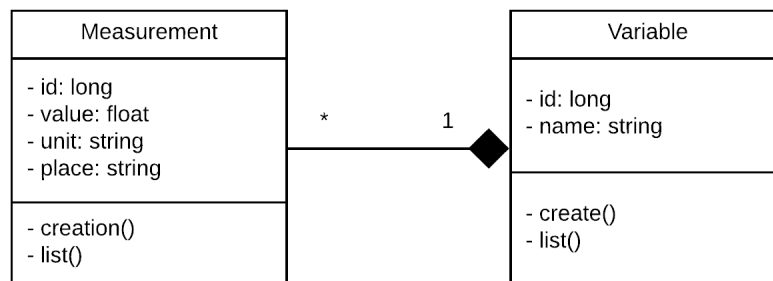


Figura 1. Modelo de datos

De acuerdo con el diagrama de despliegue (figura 2), el microservicio de variables será desplegado en una instancia Ubuntu, mientras que el de measurements en una instancia optimizada para contenedores. Adicionalmente, cada microservicio tendrá su base de datos independiente, con el objetivo de mitigar problemas como “cuellos de botella” por el uso de una única base de datos para toda la aplicación. Utilizará Kong como API Gateway, para direccionar las peticiones a cada microservicio como sea necesario. Adicionalmente, existe comunicación directa entre los dos microservicios, ya que es necesario el intercambio de información entre estos. El microservicio de variables será desplegado en una MV como en laboratorios pasados y el de measurements será desplegado como contenedor (en este taller se indica cómo crear la imagen a correr).

Existe un tercer microservicio desplegado a través de una función Serverless (en la figura referido como api-consumption) que se encargará de consumir “Measurements” en lote de un API externa y llamar al servicio de measurements para crearlas. Esta función se activa periódicamente a través de un cron (llamado Scheduler en GCP).

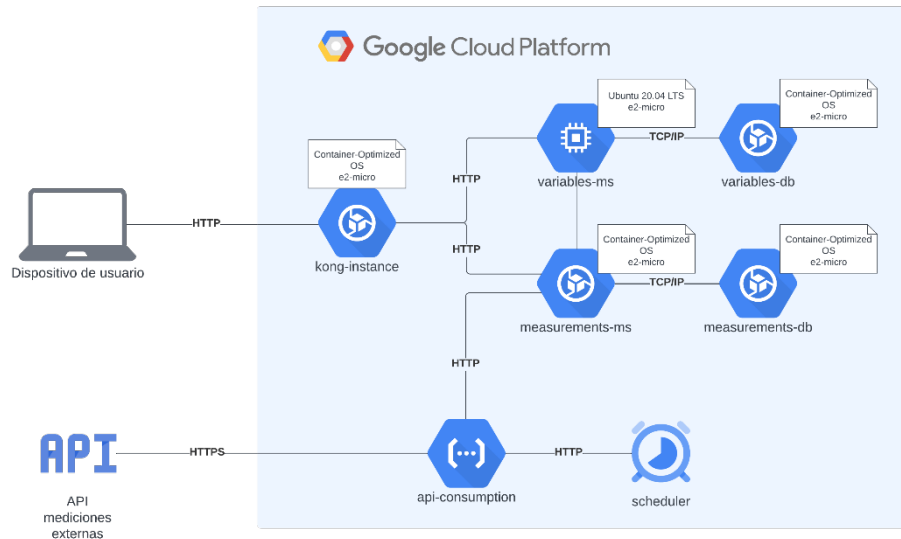


Figura 2. Vista de despliegue

5. Pre-Requerimientos

Este taller debe ser realizado **individualmente**.

5.1 Crear reglas de Firewall

Siga la [guía “del \(antes\) del laboratorio de pruebas de carga” punto 3.2](#) para la creación de un nuevo proyecto GCP. Una vez creado el proyecto cree las reglas de firewall, usando la consola gráfica de GCP (para esto también puede guiarse de la sección 3.3 de la guía “del (antes) del laboratorio de pruebas de carga”) o desde una terminal (Cloud Shell o Gcli) ejecute los siguientes comandos (recuerde reemplazar <PROJECTID> por el nombre del proyecto GCP que está usando en este laboratorio):

- ```
gcloud compute --project=<PROJECT_ID> firewall-rules create firewall-django --direction=INGRESS --priority=1000 --network=default --action=ALLOW --rules=tcp:8080 --source-ranges=0.0.0.0/0 --target-tags=rest-django
```
- ```
gcloud compute --project=<PROJECT_ID> firewall-rules create postgres-db --direction=INGRESS --priority=1000 --network=default --action=ALLOW --rules=tcp:5432 --source-tags=rest-django --target-tags=postgres-db
```
- ```
gcloud compute --project=<PROJECT_ID> firewall-rules create kong-proxy --direction=INGRESS --priority=1000 --network=default --action=ALLOW --rules=tcp:8000 --source-ranges=0.0.0.0/0 --target-tags=kong-instance
```
- ```
gcloud compute --project=<PROJECT_ID> firewall-rules create kong-admin-api --direction=INGRESS --priority=1000 --network=default --action=ALLOW --rules=tcp:8001 --source-ranges=0.0.0.0/0 --target-tags=kong-instance
```

5.2 Crear instancia DB postgres - variables-db

Cree la instancia de VM llamada *variables-db* siguiendo el tutorial de Qwiklab, [Cómo crear una máquina virtual](#), o ejecutando el comando número 9 de la [Lista de comandos GC](#). Las máquinas deben tener las siguientes características (en caso de

hacerlo con el comando 9, debe cambiar el nombre, las etiquetas de red y el PROJECT ID):

- **Nombre:** variables-db
 - **Serie:** E2
 - **Tipo de máquina:** e2-micro
 - **Disco de arranque:**
 - **Sistema operativo:** Container Optimized OS
 - **Versión:** Container-Optimized OS 101-17162.336.20 LTS
 - **Tipo de disco de arranque:** Disco persistente estándar
 - **Tamaño:** 10GB
 - **Herramientas de red:**
 - **Etiquetas de red:** postgres-db
- Conéctese a la máquina mediante la terminal SSH de Google (esto se explica en el laboratorio [Como crear una máquina virtual](#)) o a través de Putty (esto se explica en el siguiente [instructivo](#)).

El siguiente comando debe ejecutarse en la terminal de la instancia *variables-db* para el despliegue de un contenedor Postgres usando Docker.

```
>>> docker run --restart=always -d -e POSTGRES_USER=variables_user -e POSTGRES_DB=variables_db -e POSTGRES_PASSWORD=isis2503 -p 5432:5432 --name variables-db postgres
```

El comando tiene las siguientes configuraciones para el contenedor que se desplegará:

- **--restart** La opción **--restart** permite definir las políticas de reinicio del contenedor. Para este caso se envía el flag **always** el cual le indica al contenedor que al iniciar el servicio de Docker (por ejemplo, cuando se pausa la instancia y se vuelve a iniciar) el contenedor de la base de datos se reinicie automáticamente.
- **--name** La opción **--name** permite asignarle un nombre al contenedor cuando este es ejecutado.
- **-e** La opción **-e** permite pasar una variable de entorno al contenedor que se desplegará. Las variables `POSTGRES_USER`, `POSTGRES_DB` y `POSTGRES_PASSWORD` se están enviando al contenedor para definir el usuario y contraseña de la base de datos respectivamente.
- **-p 5432:5432** La opción **-p** define los puertos que expone el contenedor. El número antes de los ":" define el puerto que expone el servicio desplegado (5432 es el puerto usado por postgres) y el número después de los ":" define el puerto de la instancia en que se expone el servicio. Para este caso expondremos el servicio de postgres en el mismo puerto por default
- **postgres** Finalmente el comando finaliza con la imagen del contenedor que se desplegará. Para este caso se usará la imagen pública de postgres que puede encontrar en [Docker Hub](#).

- Al ejecutar el comando se iniciará la descarga de la imagen del contenedor, esto puede tardar unos segundos. En la consola le aparecerá lo siguiente:

```
kellygarces@variables-db ~ $ docker run -d --name postgres --env POSTGRES_USER=variables_user --env POSTGRES_DB=variables_db --env POSTGRES_PASSWORD=isis2503 -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
a603fa5e3b41: Pull complete
0207477348fd: Pull complete
16b62ca80c8f: Pull complete
fbd795da1fe1: Pull complete
9c689e39d930: Pull complete
2c441a35082c: Pull complete
1c97f440fe14: Pull complete
87a3f78bc5d1: Pull complete
264b18c6a666: Pull complete
b6e1313c492d: Pull complete
ed48cace97fa: Pull complete
a3c377e275ff: Pull complete
88fa351e30cb: Pull complete
Digest: sha256:76ed8867192b474f02e48c7b1a556d12ddfa246138ddc74bd70c891bf2873d82
Status: Downloaded newer image for postgres:latest
65387ae483b9899e9f5d8348f4f0fbed6102292fbeb6f60db7a0411fdadd4069
```

- Cuando finalice, ejecute el siguiente comando en terminal para validar el correcto levantamiento del contenedor.

```
>>> docker ps -a
```

Este comando lista los contenedores que actualmente se están ejecutando. Al ejecutar el comando deberá aparecer en consola lo siguiente:



```
af_rubriguez@variables-db: ~ $ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS
4215a8bedf09   postgres  "docker-entrypoint.sh"   34 seconds ago    Up 31 seconds    5432:5432/tcp, :::5432/tcp
af_rubriguez@variables-db: ~ $
```

5.3 Crear instancia DB postgres - measurements-db

Cree la instancia de VM llamada measurements-db siguiendo el tutorial de Qwiklab, [Cómo crear una máquina virtual](#), o ejecutando el comando número 9 de la [Lista de comandos GC](#). Las máquinas deben tener las siguientes características (en caso de hacerlo con el comando 9, debe cambiar el nombre, la etiqueta de red y el PROJECT ID):

- Nombre: measurements-db
 - Serie: E2
 - Tipo de máquina: e2-micro
 - Disco de arranque:
 - Sistema operativo: Container Optimized OS
 - Versión: Container-Optimized OS 101-17162.336.20 LTS
 - Tipo de disco de arranque: Disco persistente estándar
 - Tamaño: 10GB
 - Herramientas de red:
 - Etiquetas de red: postgres-db
- Conéctese a la máquina mediante la terminal SSH de Google (esto se explica en el laboratorio [Como crear una máquina virtual](#)) o a través de Putty (esto se explica en el siguiente [instructivo](#)).

El siguiente comando debe ejecutarse en la terminal de la instancia *measurements-db* para el despliegue de un contenedor postgres usando Docker.

```
>>> docker run --restart=always -d -e POSTGRES_USER=measurements_user  
-e POSTGRES_DB=measurements_db -e POSTGRES_PASSWORD=isis2503 -p  
5432:5432 --name measurements-db postgres
```

- Cuando finalice, ejecute el siguiente comando en terminal para validar el correcto levantamiento del contenedor.

```
>>> docker ps -a
```

5.4 Crear instancia microservicio *variables* - *variables-ms*

Cree la instancia de VM llamada *variables-ms* siguiendo el tutorial de Qwiklab, [Cómo crear una máquina virtual](#), o ejecutando el comando número 10 de la [Lista de comandos GC](#). Las máquinas deben tener las siguientes características (en caso de hacerlo con el comando 10, debe cambiar el nombre y el PROJECT ID):

- **Nombre:** variables-ms
 - **Serie:** E2
 - **Tipo de máquina:** e2-micro
 - **Disco de arranque:**
 - **Sistema operativo:** Ubuntu
 - **Versión:** 20.04 LTS (x86-64)
 - **Tipo de disco de arranque:** Disco persistente estándar
 - **Tamaño:** 10GB
 - **Herramientas de red:**
 - **Etiquetas de red:** rest-django
1. Conéctese a la máquina que acaba de crear mediante la terminal SSH de Google (esto se explica en el laboratorio [Como crear una máquina virtual](#)) o a través de Putty (esto se explica en el siguiente [instructivo](#)). Una vez conectado a la máquina realice los siguientes pasos:

```
>>> sudo apt update  
>>> sudo apt install python3-pip -y (asegúrese que está usando en  
guion medio correcto al hacer copy&paste del comando en la terminal)  
>>> export LC_ALL=C
```
 2. Clone el repositorio de la aplicación e ingrese al microservicio de variables en la carpeta *variables*. Luego instale los requerimientos para el funcionamiento del microservicio.

```
>>> git clone https://github.com/ISIS2503/ISIS2503-Microservices-  
AppDjango.git  
>>> cd ISIS2503-Microservices-AppDjango/variables  
>>> pip3 install -r requirements.txt
```
 3. Si tuvo problemas al instalar alguna de las dependencias del requirements.txt, intente instalar cada una por separado, por ej., para psycopg2-binary sería:

```
>>> pip3 install psycopg2-binary==2.8.6
```

- Actualice el archivo **settings.py** del proyecto para configurar la conexión a la instancia de la base de datos del microservicio correspondiente creada anteriormente. Para esto abra el archivo con el comando que le damos a continuación y luego asegúrese que el apartado “DATABASES” quede como se muestra en la captura que sigue. Note que la IP que va en HOST va a cambiar cada vez que usted reinicie la instancia donde quedó la base de datos, coloque la **IP interna**.

```
>>> sudo nano monitoring/settings.py
```

```
# Database
# https://docs.djangoproject.com/en/2.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql_psycopg2',
        'NAME': 'variables_db',
        'USER': 'variables_user',
        'PASSWORD': 'isis2503',
        'HOST': '10.128.0.22',
        'PORT': '5432',
    }
}
```

- Guarde los cambios presionando las teclas **ctrl + x**, luego la tecla **y**, por último presione la tecla **Enter**.
- Ejecute las migraciones a la base de datos. Solo es necesario hacerlo en una de las instancias que creó.

```
>>> python3 manage.py makemigrations
>>> python3 manage.py migrate
```

- Finalmente ejecute el siguiente comando para ejecutar el microservicio.

```
>>> python3 manage.py runserver 0.0.0.0:8080
```

5.5 Crear instancia microservicio *measurements* - *measurements-ms*

Cree la instancia de VM llamada *measurements-ms* siguiendo el tutorial de Qwiklab, [Cómo crear una máquina virtual](#), o ejecutando el comando número 10 de la [Lista de comandos GC](#). Las máquinas deben tener las siguientes características (en caso de hacerlo con el comando 10, debe cambiar el nombre y el PROJECT ID):

- **Nombre:** measurements-ms
- **Serie:** E2
- **Tipo de máquina:** e2-micro
- **Disco de arranque:**
 - **Sistema operativo:** Container Optimized OS
 - **Versión:** Container-Optimized OS 101-17162.336.20 LTS
 - **Tipo de disco de arranque:** Disco persistente estándar
 - **Tamaño:** 10GB
- **Herramientas de red:**

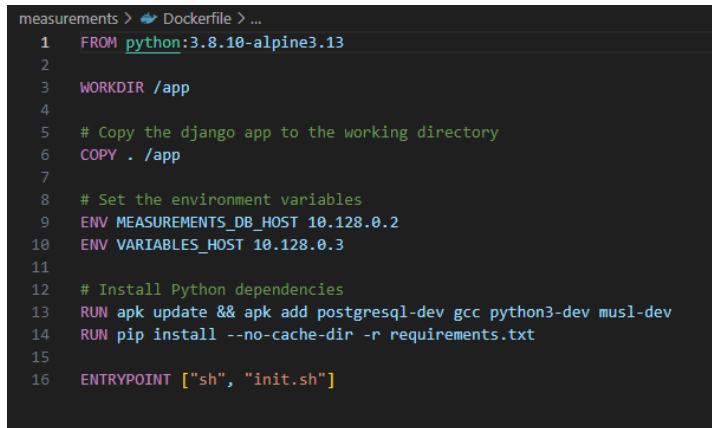
○ **Etiquetas de red: rest-django**

1. Conéctese a la máquina por SSH y ejecute los siguientes comandos:

```
>>> git clone https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango.git
>>> cd ISIS2503-Microservices-AppDjango/measurements
```

2. Ya en el directorio de measurements usted podrá crear la imagen del servicio para Docker. La creación de la imagen es un proceso sencillo pues el repositorio ya incluye el Dockerfile para crear la imagen. Este archivo contiene las instrucciones para construir la imagen como tomar como base otra imagen, configurar variables o ejecutar comandos.

- Visualice el archivo Dockerfile dentro de measurements. Podrá ver un contenido como la siguiente imagen.



```
measurements > Dockerfile > ...
1 FROM python:3.8.10-alpine3.13
2
3 WORKDIR /app
4
5 # Copy the django app to the working directory
6 COPY . /app
7
8 # Set the environment variables
9 ENV MEASUREMENTS_DB_HOST 10.128.0.2
10 ENV VARIABLES_HOST 10.128.0.3
11
12 # Install Python dependencies
13 RUN apk update && apk add postgresql-dev gcc python3-dev musl-dev
14 RUN pip install --no-cache-dir -r requirements.txt
15
16 ENTRYPOINT ["sh", "init.sh"]
```

Contenido de Dockerfile

Dentro del Dockerfile están varios comandos que guían la creación de la imagen. Las descripciones de los comandos usados se encuentran en la siguiente tabla.

Comando	Parámetros	Descripción
WORKDIR	Ruta de directorio. Ej: /app	Se crea el directorio indicado en la imagen y se le indica a Docker que los siguientes comandos se ejecutaran dentro de esta ruta.
COPY	• Ruta de directorio/archivo origen. Ej: . • Ruta de directorio/archivo destino. Ej: /app	Docker copia el archivo o directorio origen desde el host (su computador por ejemplo) y lo copia a la ruta destino dentro de la imagen.
ENV	• Nombre de la variable. Ej: VARIABLES_HOST	Configura las variables de ambiente de la imagen con su nombre y valor. Estas son útiles

	• Valor de la variable. Ej: 10.128.0.2	dentro de los contenedores.
RUN	Comando a ejecutar dentro de WORKDIR. Ej: apk update	Ejecuta el comando especificado dentro de la imagen. Sólo se ejecuta al momento de la creación de la imagen.
ENTRYPOINT	Lista con un comando y los argumentos. Ej: ["sh", "init.sh"]	Se especifica el comando que el contenedor ejecutará una vez creado o iniciado.
FROM	Imagen base. Ej: python:alpine	Se toma como base la imagen indicada para aprovechar su software o configuraciones.

En síntesis, el Dockerfile creará una imagen basada en python 3.8.10 con el repositorio actual. Luego, configurará las variables de ambiente (IPs del microservicio de variables y de la DB de measurements) e instalará las dependencias del proyecto. Y, por último, indica qué se debe ejecutar dentro del contenedor cuando inicie.

- En ENTRYPOINT se indica a Docker que se debe ejecutar el archivo init.sh al iniciar el contenedor. Ese archivo contiene los comandos de la siguiente imagen.

```

measurements > $ init.sh
1  #!/bin/bash
2
3  python3 manage.py makemigrations
4
5  python3 manage.py migrate
6
7  python3 manage.py runserver 0.0.0.0:8080
  
```

Contenido de init.sh

Los comandos son exactamente los mismos que usted ha usado siempre que quiere correr el proyecto Django de los talleres del curso. Primero se crean las migraciones del proyecto, luego se migran a la base de datos y se finaliza ejecutando el servidor web.

- Proceda a la creación de la imagen usando la terminal SSH de la MV y ejecutando este comando:

```
>>> docker build -t measurements-app .
```
- Cuando finalice el proceso ya tendrá localmente en la máquina la imagen del servicio de measurements bajo el nombre de "measurements-app". Esto lo usará al correr un contenedor.

3. Finalice corriendo el contenedor Docker que ejecutará el microservicio de measurements.

- Ejecute este comando cambiando antes las direcciones IP **privadas** por las de su despliegue.

```
>>> docker run --name measurements -it --restart=on-failure:3 -e MEASUREMENTS_DB_HOST=<ip-measurements-db> -e VARIABLES_HOST=<ip-variables-ms> -p 8080:8080 measurements-app
```
- Podrá ver que la terminal muestra los mismos mensajes que cuando intenta correr Django sin docker.

```
javelino2311@docker: ~/ISIS2503-Microservices-AppDjango/measurements $ docker run --name measurements -it -e MEASUREMENTS_DB_HOST=10.128.0.16 -e VARIABLES_HOST=10.128.0.17 -p 8080:8080 app-ms
Migrations for 'measurements':
  measurements/migrations/0001_initial.py
    - Create model Measurement
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, measurements, sessions
Running migrations:
  No migrations to apply.
Performing system checks...

System check identified no issues (0 silenced).
November 19, 2023 - 00:44:54
Django version 2.1.5, using settings 'monitoring.settings'
Starting development server at http://0.0.0.0:8080/
Quit the server with CONTROL-C.
```

Mensajes del terminal luego ejecutar docker run.

NOTA:

- Compare el despliegue de los microservicios variables y measurements. ¿Qué pros y contras encuentra en ambas formas de despliegue?

Finalice pausando todas las instancias hasta el día del laboratorio.